sqlmap: integration and how to use

Updated: 2 November 2026 (Cairo) · sqlmap 1.9.6 stable (public capture) · sqlmap 1.10.11.2 development (local check)

> **Authorization first.** Test only systems you own or have written permission to test. Even SQL injection detection can change data or trigger security controls. Do not run sqlmap on any site unless its operator allows that specific test. In this project, the only public site allowed is SekuMart's product `id` parameter.

1. What was added

File	Role
`backend/app/ai_engine/pentest/tools/sqlmap_adapter.py`	Runs the real sqlmap program. Uses a private request file and output folder for each scan, and removes them after the scan.
`backend/app/ai_engine/pentest/modules/tools/sqlmap_scanner.py`	The framework module `sqlmap_scanner`. Checks scope and impact, then records findings.
`backend/tests/unit/test_sqlmap_integration.py`	Adapter, safety, error, cleanup and concurrency tests
`scripts/live_tests/sqlmap_framework_smoke.py`	Local lab with real sqlmap, plus real Nmap if installed. Its Metasploit endpoint is a **fake**.
`scripts/live_tests/beginner_tools_lab.py`	`sqlmap` mode for the beginner lab
`backend/app/core/config.py` and `backend/.env.example`	`SQLMAP_ENABLED`, `SQLMAP_PATH`, `SQLMAP_TIMEOUT`

2. What it can do

Capability	Current behavior
Target	One approved HTTP or HTTPS URL per run
Request method	GET parameters, or URL-encoded form POST (POST needs at least `limited` impact)
Parameters	All supported parameters, or only the ones you select with `sqlmap_parameters`
Techniques	Boolean-based and error-based (`BE`, `B` or `E`)
Risk	Fixed at 1
Level	1 by default. Levels 2–5 need at least `limited` impact.
Authentication	Approved headers, token or cookies. Credentials are not put on the command line.
Rate	One sqlmap HTTP thread. Delay is at least the framework's delay.
Redirects	Not followed
Output	A confirmed injection becomes a **HIGH** finding (CWE-89). A negative is **INFO** only when every selected parameter was tested. An incomplete scan is an **ERROR**.

Not provided: arbitrary sqlmap flags, crawling, JSON bodies, database dumping, shells, file access, stacked queries, time-based and UNION techniques.

3. Install sqlmap

Mac (zsh):

```
brew install sqlmap
sqlmap --version
```

Windows (PowerShell): Install Git for Windows if `git --version` fails. Then:

```
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git "$HOME\sqlmap"
.\backend\.venv\Scripts\python.exe "$HOME\sqlmap\sqlmap.py" --version
$env:SQLMAP_PATH = "$HOME\sqlmap\sqlmap.py"
```

The `$env:SQLMAP_PATH` line applies only to the current PowerShell window. Repeat it in a new window.

4. Settings

In `backend/.env` (private; do not commit):

```
SQLMAP_ENABLED=true
SQLMAP_PATH=sqlmap
SQLMAP_TIMEOUT=90
```

On Windows with a clone, set `SQLMAP_PATH` to the full path of `sqlmap.py`. Restart the backend after changing `.env`. If sqlmap is missing or disabled, the module returns an INFO finding instead of crashing.

The framework's outer `PENTEST_MODULE_TIMEOUT` (120 s by default) includes the version check and the scan. If an approved scan needs more time, raise both `SQLMAP_TIMEOUT` and `PENTEST_MODULE_TIMEOUT`.

5. Verify on your computer (local lab)

Mac (zsh):

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py sqlmap
./backend/.venv/bin/python scripts/live_tests/sqlmap_framework_smoke.py
```

Windows (PowerShell):

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py sqlmap
.\backend\.venv\Scripts\python.exe scripts\live_tests\sqlmap_framework_smoke.py
```

Expected from the beginner lab: `PASS: real sqlmap detected the lab's intentional SQL injection in id.`

Expected from the smoke test: several `PASS` lines, including `real sqlmap detected GET SQLi`, `parameterized GET control produced no actionable finding`, `real sqlmap detected SQLi in an authorized form POST`, and `two real sqlmap scans ran concurrently in isolated engagements`. The smoke test reports Metasploit as fake.

Both checks use a temporary local lab on `127.0.0.1`. They do not test any public site.

6. Use it from Python

Save this as `sqlmap_check.py` in the **backend** folder. Replace the URL only with a target you are authorized to test. The example uses the public practice site's allowed parameter.

```
import asyncio
from app.ai_engine.pentest import ExecutionEngine, PentestContext, ScopeConfig, default_registry

async def main():
    engine = ExecutionEngine(registry=default_registry())
    context = PentestContext(
        target="https://sekumart.sekuripy.hr/product.php?id=1",
        scope=ScopeConfig(
            allowed_targets=["sekumart.sekuripy.hr"],
            allowed_ports=[443],
            max_impact="proof_of_concept",
            request_delay=0.25,
        ),
        config={
            "sqlmap_parameters": ["id"],
            "sqlmap_techniques": "BE",
            "sqlmap_level": 1,
            "sqlmap_timeout": 90,
        },
    )
    findings = await engine.execute("sqlmap_scanner", context)
    for finding in findings:
        print(finding.severity.value, finding.title)

asyncio.run(main())
```

Run it from the backend folder:

Mac (zsh): `./.venv/bin/python sqlmap_check.py`

Windows (PowerShell): `.\.venv\Scripts\python.exe sqlmap_check.py`

For a form POST, set `max_impact` to `limited` or higher and add `"sqlmap_method": "POST"` and `"sqlmap_data": "id=1"` to `config`, using test data you are authorized to send.

7. Public result (the only public sqlmap capture)

Field	Value
Target	`https://sekumart.sekuripy.hr/product.php?id=1`
Parameter tested	`id` (GET only)
Captured	2026-10-28, 01:53 UTC
sqlmap version	1.9.6#stable
Result	Error-based SQL injection reported in GET `id` (MySQL / MariaDB)

The capture script is `scripts/live_tests/public_tool_results.py sqlmap --confirm-public-tests`. It enforces its own rate limit. Run it only when a fresh capture is needed, and not for other sites.

8. Limits

- Metasploit is not part of this integration. Nmap results are not passed to sqlmap automatically.
- The API, AI agent and interface do not start sqlmap yet.
- Testing window is not enforced. Schedule tests outside the window yourself.
- Ordinary DNS names are not pinned.
- Results include the target URL. Do not put secrets in URLs.
- The Mac and Windows install steps were not run on those systems in this project.